Multi-Agent Education System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	AMAN VERMA	EN22CS301110
02	ANJALI JAISWAL	EN22CS301132
03	ANKIT PARMAR	EN22CS301135
04	ANSH LAAD	EN22CS301141
05	APOORVA GUPTA	EN22CS301190
06	ARYA BHAWSAR	EN22CS301209

Group Name:12D2

Project Number:AAI-24

Industry Mentor Name:

University Mentor Name:Ajeet Singh Rajput

Academic Year:2025-26

Problem Statement & Objectives

1.Problem Statement

Traditional AI systems rely on a single model to process user queries, which often leads to **unstructured, incomplete, or less accurate responses**. These systems lack specialization and struggle to handle complex educational queries that require **research, reasoning, and structured explanation**.

Additionally, most AI systems do not support **collaboration between multiple intelligent agents**, limiting their ability to generate high-quality, well-organized, and context-aware educational content.

To overcome these limitations, there is a need for a **Multi-Agent Education System** that uses **Agentic AI architecture**, where multiple specialized agents collaborate to process queries, generate structured content, and enhance learning with relevant outputs.

2. Project Objectives

* To design a **multi-agent system** where different agents perform specialized tasks (research,content generation, visualization).
* To implement an **orchestrator agent** that manages communication and workflow between agents.
* To generate **structured and high-quality educational content** using LLMs.
* To improve **accuracy and clarity** by dividing tasks among multiple agents.
* To support **real-time response generation** without storing user data (stateless system).
* To enhance learning by providing **text explanations along with visual aids (images/diagrams)**.
* To ensure **scalability and modularity** of the system for future enhancements

3. Scope of the Project

The scope of the Multi-Agent Education System includes:

- Processing **education-related user queries** only.
- Generating **structured textual explanations** using LLM APIs.
- Producing **visual content (images/diagrams)** through an image generation agent.
- Managing **agent communication and workflow** using frameworks like CrewAI or LangChain.
- Operating as a **stateless system**, where:
    - No user data is stored
    - No conversation history is maintained
- Providing **real-time responses** based on user input.

Proposed Solution

1. Key features

***Multi-Agent Collaboration** – Multiple intelligent agents work together instead of a single AI model

* **Orchestrator-Based Control** – Central agent manages task flow and decision-making

* **Intent Detection System** – Filters whether the query is education-related

* **Task Specialization** –

- Research/Content Agent → Generates explanations
- Image Agent → Creates diagrams

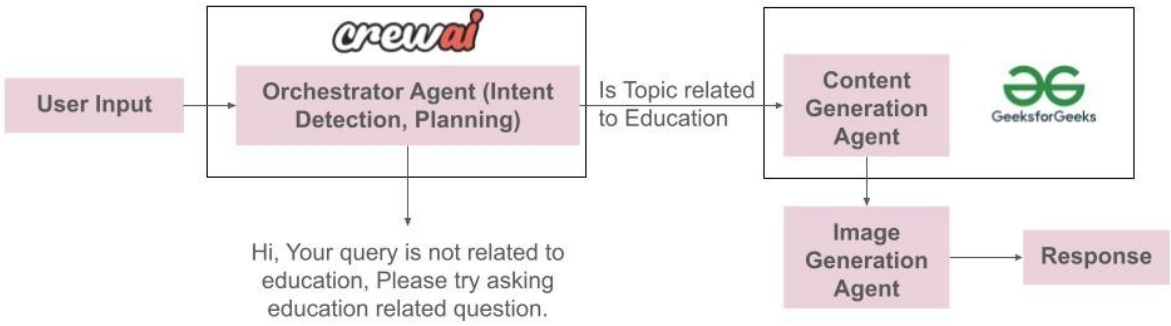
* **Structured Output Generation** – Produces well-organized and readable content

* **Real-Time Processing** – Generates responses instantly using APIs

* **Stateless Architecture** – No data storage, ensuring privacy and simplicity

* **Extensible Design** – Easily add new agents (e.g., Web Search Agent in future)

2. Overall Architecture / Workflow



3. Tools & Technologies Use

Core Technologies

- **Python** → Main programming language
- **CrewAI** → Multi-agent orchestration
- **LangChain** → Tool integration and agent design

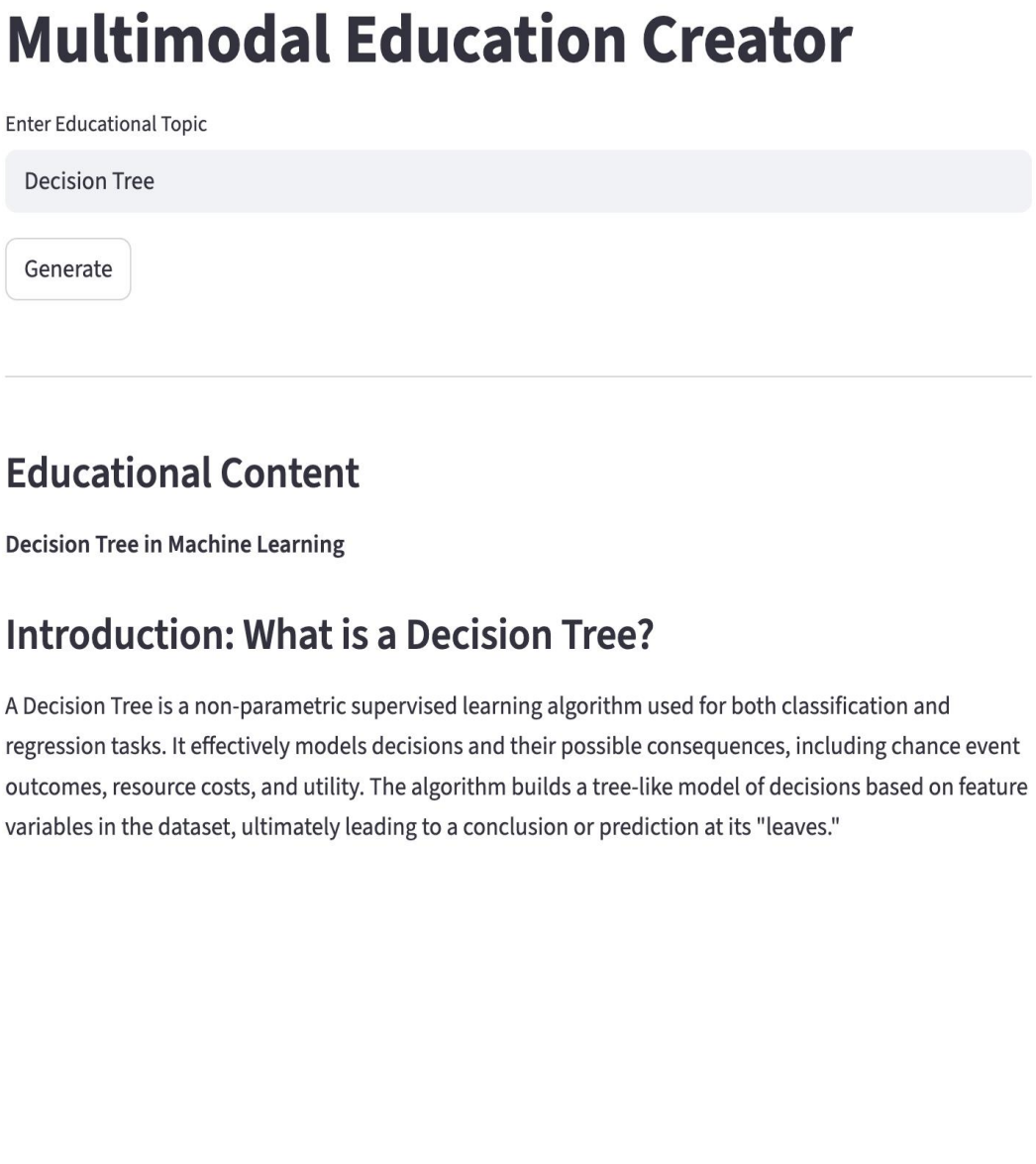
AI Components

- **LLM APIs** (e.g., GPT models)
  - Content generation
  - Reasoning and formatting
- **Image Generation Models**
  - Create diagrams/visuals
  - Enhance learning experience

Results & Output

Add the below details here:

1. *Screenshots / outputs*



2. *Reports / dashboards / models*

At its core, a Decision Tree is a flowchart-like structure. Each internal node within this structure represents a "test" on an attribute (for example, "Is the outlook sunny?"). Each branch stemming from an internal node represents the outcome of that test, and each leaf node represents a final class label (in classification tasks) or a numerical value (in regression tasks), or a decision. The paths traced from the root of the tree down to a leaf node collectively represent classification rules.

The fundamental idea behind decision trees is to recursively divide the data into smaller, more manageable subsets. This division is based on the values of the input features, continuing until each subset contains data points that are as homogeneous as possible with respect to the output variable (the target).

2. How it Works (Intuition)

To grasp the intuition behind Decision Trees, imagine you're trying to decide whether to play tennis. Your decision process likely involves considering several factors, such as the weather outlook, temperature, humidity, and wind. A decision tree mimics this human decision-making process by following a similar sequential approach:

- **Step 1:** You first look at the most important factor (e.g., Outlook).
- **Step 2:** If the Outlook is Rainy, you then consider another factor (e.g., Wind).
- **Step 3:** Based on the Wind, you make a final decision (e.g., if Windy, don't play; if Not Windy, play).

The Decision Tree algorithm learns these "rules" and the order of importance of factors directly from historical data.

3. Key Terminology

Understanding the specific terminology is crucial for comprehending Decision Trees:

- **Root Node:** This is the topmost node in the tree. It represents the entire dataset before any splits occur and is the starting point for all decisions.
- **Internal Node (Decision Node):** Any node that has two or more branches extending from it. It represents a test or condition on an attribute, and each branch from it represents one of the possible outcomes of that test.
- **Leaf Node (Terminal Node):** A node that does not split further. It signifies the final decision or prediction, such as a class label in classification or a numerical value in regression.
- **Splitting:** The process of dividing a node into two or more sub-nodes based on a chosen feature and a specific split criterion.
- **Branch/Sub-tree:** A section of the entire tree, representing a specific decision path or a smaller tree structure within the main tree.
- **Parent Node:** A node that is divided into one or more sub-nodes.
- **Child Node:** The sub-nodes that branch out directly from a parent node.
- **Pruning:** The process of removing sub-nodes of a decision node. It is the opposite of splitting and is primarily used to reduce the complexity of the tree and prevent overfitting to the training data.

4. Splitting Criteria (Attribute Selection Measures)

To construct the tree, at each internal node, the algorithm must decide which attribute to split on. This decision is made using various metrics, known as Attribute Selection Measures (ASM), which evaluate the "purity" or "homogeneity" of a node. The primary objective is to select the split that results in the most homogeneous (or "pure") child nodes, meaning data points within each child node are very similar with respect to the target variable.

The most common criteria include:

- **Gini Impurity (for Classification):**
  - Measures how often a randomly chosen element from the set would be incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset.
  - A Gini impurity of 0 means all elements belong to a single class (perfectly pure node). A Gini impurity of 0.5 (for a binary classification) indicates maximum impurity, meaning elements are equally distributed across classes.
  - The algorithm chooses the split that minimizes the weighted average Gini impurity of the child nodes.
  - Formula for a node *t*: $Gini(t) = 1 - \sum_{i=1}^{c}(P_i)^2$, where P_i is the probability of class *i* at node *t*, and *c* is the number of classes.

5. Algorithm Steps (Simplified)

The process of building a Decision Tree can be broken down into these recursive steps:

1. **Start at the Root Node:** Begin the tree construction process with the entire training dataset at the root node.
2. **Select the Best Attribute:** Use an Attribute Selection Measure (such as Gini impurity, Information Gain, or Variance Reduction) to evaluate all available attributes and find the one that offers the best split. The "best" attribute is defined as the one that creates the most homogeneous child nodes.
3. **Create Child Nodes:** Split the dataset into two or more subsets based on the distinct values or ranges of the chosen "best" attribute. Each subset forms a new child node.
4. **Recursion:** Repeat steps 2 and 3 for each newly created child node. This recursive process continues until certain stopping conditions are met.
5. **Stopping Condition:** The splitting process halts when one or more of the following conditions are met:
   - All data points within a node belong to the same class (indicating a perfectly pure node).
   - There are no remaining attributes to split on, or all remaining attributes have already been used.
   - A pre-defined stopping criterion is met, such as reaching a maximum tree depth, requiring a minimum number of samples to split a node, or having a minimum number of samples in a leaf node.
6. **Assign Leaf Nodes:** Once a stopping condition is met for a particular branch, that node becomes a leaf node. Its label or value is assigned based on the majority class of the samples within that node (for classification) or the average value of the samples (for regression).

6. Pros of Decision Trees

Decision Trees offer several compelling advantages:

- **Simple to Understand and Interpret:** The tree structure is highly intuitive and can be easily visualized. This makes the decision-making logic transparent and comprehensible, even for non-technical users.
- **Minimal Data Preparation:** They require less data cleaning and preprocessing compared to some other algorithms. Decision trees can handle both numerical and categorical data without extensive feature scaling or normalization.
- **Non-parametric:** They make no assumptions about the underlying distribution of the data, which makes them flexible for various datasets.
- **Handles Multi-output Problems:** Decision trees can be extended to predict multiple output variables simultaneously.
- **Implicit Feature Selection:** Features that are more important for classification or regression naturally appear higher up in the tree, effectively performing an implicit form of feature selection.
- **Basis for Ensemble Methods:** They serve as the fundamental building blocks for more powerful and robust ensemble learning techniques like Random Forests and Gradient Boosting.

7. Cons of Decision Trees

Despite their advantages, Decision Trees also have limitations:

- **Prone to Overfitting:** Without proper regularization or pruning, a decision tree can create overly complex trees that fit the training data too closely. This can lead to poor generalization performance on unseen data.
- **Instability:** Small variations in the training data can sometimes result in a completely different tree structure. This sensitivity arises because the greedy approach to splitting can lead to different initial splits influencing subsequent branches significantly.
- **Local Optima:** The greedy algorithm used to build decision trees selects the best split at each step without looking ahead. This means it may converge to a locally optimal tree rather than a globally optimal one, potentially missing a better overall tree structure.
- **Bias Towards Dominant Classes:** If the dataset contains imbalanced classes, decision trees can be biased towards the majority class. It is often recommended to balance the dataset prior to fitting a decision tree or use techniques like class weighting.
- **Complexity for Continuous Variables:** Splitting on continuous variables can be computationally intensive as it requires evaluating many possible split points to find the optimal threshold.

8. Use Cases

Decision Trees are versatile and applied in various domains, including:

- **Medical Diagnosis:** Assisting in identifying diseases based on patient symptoms and medical test results.
- **Customer Churn Prediction:** Predicting which customers are most likely to discontinue a service or subscription.
- **Credit Risk Assessment:** Evaluating the creditworthiness of loan applicants based on their financial history and other attributes.
- **Fraud Detection:** Identifying potentially fraudulent transactions in financial or insurance industries.
- **Predicting Product Demand:** Estimating future sales for specific products based on historical data and market trends.

9. Example (Conceptual)

Let's illustrate a simplified Decision Tree to decide if a person will buy a product based on their Age, Income, and Education level:

```
        [Root Node]
           Age <= 30?
          /         \
        Yes         No
       /             \
    [Node 1]          [Node 2]
    Income <= 50k?    Education = High?
    /      \          /          \
  Yes      No        Yes         No
  /         \        /           \
 Buy      Don't Buy  Don't Buy   Buy
 (Leaf)   (Leaf)     (Leaf)      (Leaf)
```

In this conceptual tree:

- The process starts at the **Root Node** by evaluating the 'Age' attribute.
- If 'Age' is less than or equal to 30, the path leads to **Node 1**, which then checks 'Income'.
- If 'Income' is less than or equal to 50k, the prediction is 'Buy' (a **Leaf Node**).
- Conversely, if 'Age' is greater than 30, the path goes to **Node 2**, checking 'Education'.
- If 'Education' is 'High', the prediction is 'Don't Buy' (another **Leaf Node**).

10. Relationship to Ensemble Methods

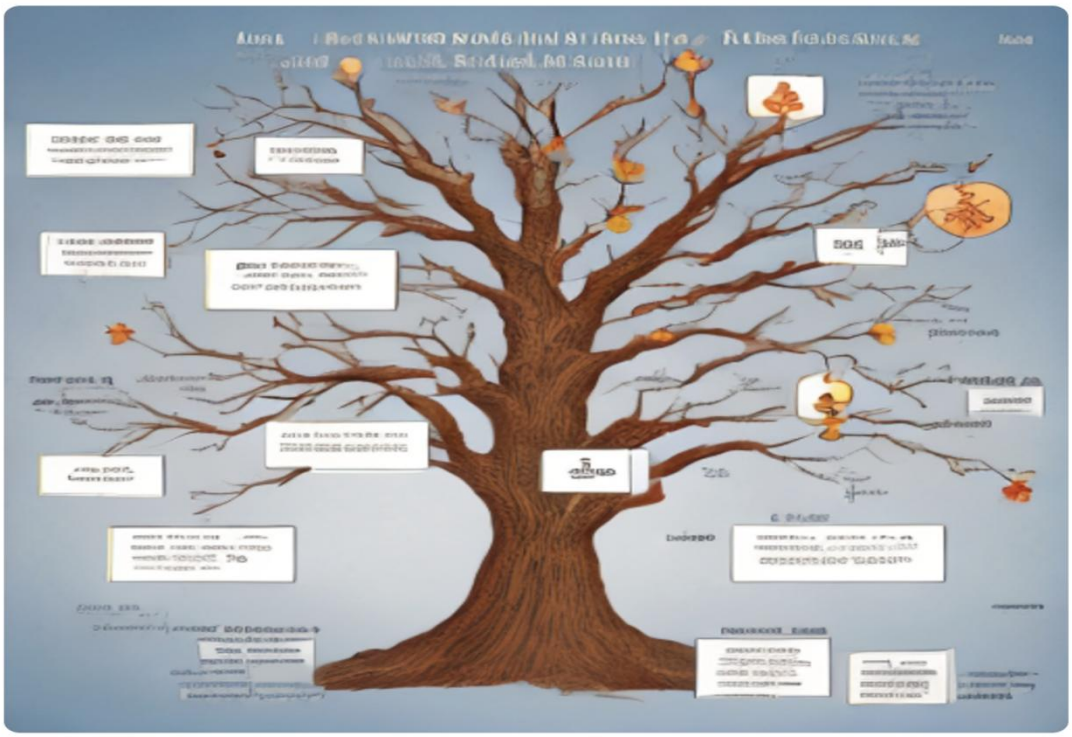
Decision Trees serve as the foundational components for some of the most powerful and widely used ensemble learning methods:

- **Random Forests:** This ensemble technique constructs multiple decision trees. Each tree is trained on a random subset of the data (bootstrapping) and considers only a random subset of features at each split. The final prediction is derived by averaging the predictions of all individual trees (for regression) or by taking a majority vote (for classification). This collective approach significantly reduces overfitting and improves the model's robustness and accuracy.
- **Gradient Boosting (e.g., XGBoost, LightGBM):** In contrast to Random Forests, Gradient Boosting builds trees sequentially. Each new tree in the sequence is trained to correct the errors (or residuals) made by the combination of all previously built trees. It iteratively focuses on the instances that were misclassified or had larger residuals, gradually improving the model's overall accuracy and performance.

Conclusion

Decision Trees are fundamental and intuitive machine learning algorithms that provide a clear, interpretable way to make predictions. They are non-parametric supervised learning models used for both classification and regression, characterized by their flowchart-like structure that guides decisions from a root node to leaf nodes through sequential splits based on feature attributes. While they are easy to understand, require minimal data preparation, and offer implicit feature selection, they are also prone to overfitting and instability if not properly managed through techniques like pruning. Despite these limitations, their simplicity and effectiveness make them a crucial component and a foundational building block for more powerful ensemble techniques like Random Forests and Gradient Boosting, which significantly enhance their performance and robustness.

Educational Diagram



Key Outcomes

- Successfully designed and implemented a **Multi-Agent Education System** using Agentic AI concepts
- Demonstrated effective **collaboration between multiple specialized agents** (Orchestrator, Content, Image Agent)
- Achieved **structured and high-quality educational content generation**
- Implemented **intent detection** to filter and validate user queries
- Developed a **stateless architecture**, ensuring **high data privacy and security**
- Enabled **real-time response generation** using LLM APIs
- Improved **clarity of learning** through integration of **text + visual outputs**
- Built a **modular and scalable system** that can be easily extended with new agents
- Gained practical experience with **modern AI frameworks (CrewAI, LangChain)**
- Established a strong foundation for **future enhancements like web search, personalization, and memory systems**

Conclusion

The **Multi-Agent Education System** demonstrates how **Agentic AI architecture** can significantly improve the quality of educational content generation. By dividing tasks among multiple specialized agents—such as the Orchestrator, Content Generation Agent, and Image Generation Agent—the system produces **structured, accurate, and visually enhanced outputs**.

Unlike traditional single-model AI systems, this approach ensures **better organization, scalability, and efficiency**. The stateless design also guarantees **high data privacy**, as no user information is stored. Overall, the system successfully achieves its goal of providing **intelligent, real-time, and high-quality educational assistance**.

Future Scope & Enhancements

The system can be further improved and expanded in the following ways:

1. Integration of Web Search Agent

- Add real-time data fetching using tools like SerpAPI

- Helps provide **latest information and current trends**

2. Personalized Learning

- Introduce user profiles and history tracking
- Provide **customized learning recommendations**

3. Memory & Context Awareness

- Implement short-term or long-term memory
- Enable **context-aware conversations**

4. Advanced Visualization

- Generate more detailed diagrams, charts, and animations
- Improve understanding of complex topics

5. Multi-Language Support

- Support regional and international languages
- Make the system accessible to a wider audience

6. Voice-Based Interaction

- Add speech-to-text and text-to-speech features
- Enable hands-free learning experience

7. Performance Optimization

- Introduce caching for repeated queries
- Reduce API calls and improve response time

8. Deployment as Web/App Platform

- Convert into a full-fledged:
  - Web application
  - Mobile app
- Improve usability and accessibility

9. Enhanced Agent Collaboration

- Add more specialized agents:
  - Quiz Generator Agent
  - Doubt Solver Agent
  - Assessment Agent